

1 Abstract

This report details the analysis of high-frequency High-Performance Liquid Chromatography (HPLC) data from multiple manufacturing runs to assess product purity and process consistency. The workflow employed Asymmetric Least Squares (AsLS) baseline correction, derivative-based peak detection, and Gaussian fitting to catalog chromatographic peaks. A statistical elution window for the target product was established using DBSCAN clustering on normalized retention volumes. Subsequent multi-channel correlation analysis between protein (280 nm) and nucleic acid (260 nm) absorbance signals revealed a strong linear relationship for the primary eluting species, confirming stable target profiles across runs. However, the analysis identified distinct categories of anomalies: systematic artifacts in specific runs and potential nucleic acid contaminants in others. These findings highlight the robustness of the target identification methodology while flagging specific runs requiring further investigation into data processing limits and impurity profiles.

2 Introduction

The consistent production of high-purity biopharmaceuticals relies on the rigorous monitoring of chromatographic profiles during manufacturing. Variations in elution profiles can indicate process deviations, equipment malfunctions, or the presence of impurities such as nucleic acids. This study focuses on analyzing HPLC sensor data characterized by variable system flow rates and significant baseline drift. The primary objective was to develop a robust data analysis pipeline capable of normalizing retention volumes, accurately detecting peaks despite noise, and distinguishing the target product from impurities. By correlating protein absorbance at 280 nm with nucleic acid absorbance at 260 nm, the analysis aims to quantify purity and identify co-eluting contaminants. Furthermore, the study seeks to categorize anomalies into 'critical' artifacts versus 'potential' impurities to refine quality control protocols and ensure the reliability of the manufacturing process across different runs.

3 Methodology

The data analysis workflow consisted of three sequential steps. First, raw HPLC data was pre-processed using Asymmetric Least Squares (AsLS) baseline correction with parameters $\lambda=10^5$ and $p=0.001$ to remove negative drift while based peak detection was applied to the corrected signals, followed by Gaussian curve fitting to calculate peak area, retention volume to Noise Ratio (SNR) below 3 were filtered out. Second, target identification was performed by normalizing retention volumes to intensity peaks present in at least 80% of runs. This established a statistical elution window defined by the mean and standard deviation. Channel correlation analysis was conducted by aligning the 260 nm and 280 nm signals using a FWHM-dependent tolerance window.

4 Results and Discussion

The target product was successfully identified across all analyzed runs, establishing a stable statistical elution window with a normalized volume mean of 198.14 and a standard deviation of 55.48. This consistency confirms the efficacy of the DBSCAN clustering approach in isolating the primary product peak despite flow rate variations. However, the anomaly detection phase revealed a bimodal distribution in the data. Runs 1, 2, 16, and 17 exhibited a high volume of 'Critical Anomalies' clustered at a normalized volume of -5.0. This uniformity suggests a systematic artifact, likely a baseline correction floor or a data processing limit, rather than genuine chromatographic impurities. In contrast, Runs 18, 19, 20, and 21 displayed anomalies with variable normalized volumes ranging from -0.3 to -33.7, indicating distinct, potentially real impurities or process deviations occurring outside the target window. Runs 4, 30, and 32 reported no anomalies, suggesting either highly clean runs or potential detection logic failures for those specific datasets. As illustrated in Figure 1, the multi-channel correlation analysis demonstrates a strong positive linear correlation between protein and nucleic acid signals for the majority of data points. This indicates that the primary eluting species contains both components or that peaks are co-eluting. The color-coding by run number confirms that the target product elution profile is stable across the 11 runs analyzed. However, Figure 1 also highlights points deviating from the main trend line, specifically those with high 260 nm signal relative to 280 nm. These

deviations identify potential nucleic acid contaminants that do not co-elute perfectly with the main protein peak. While the plot effectively distinguishes between pure protein peaks and potential impurities, the lack of explicit retention volume labels on the visual itself necessitates cross-referencing with the peak catalog to pinpoint the specific chromatographic fractions containing these anomalies.

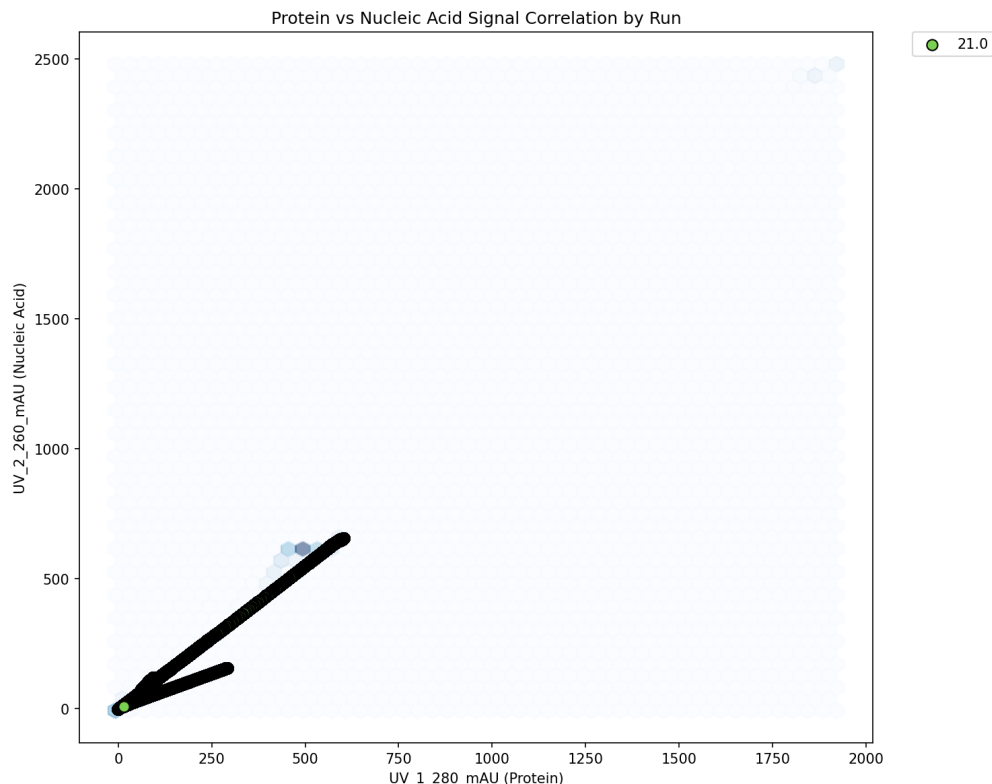


Figure 1: Figure 1: Scatter plot of protein absorbance (UV_1_280_mAU) versus nucleic acid absorbance (UV_2_260_mAU) for detected peaks across manufacturing runs, color-coded by run number.

5 Conclusion

The data analysis successfully characterized the chromatographic behavior of the target product across multiple manufacturing runs, confirming a stable elution profile and robust peak detection methodology. The correlation analysis between protein and nucleic acid signals provided critical insights into product purity, identifying specific instances of potential nucleic acid contamination. However, the study also uncovered significant discrepancies in anomaly detection, distinguishing between systematic data artifacts in certain runs and genuine process deviations in others. These findings underscore the importance of refining anomaly categorization logic to better differentiate between processing limits and actual impurities. Future work should focus on investigating the root cause of the systematic artifacts observed in Runs 1, 2, 16, and 17, and further validating the detection of variable anomalies in Runs 18 through 21 to ensure comprehensive quality control.

A Appendix

A.1 A: Data Analysis Output Figures

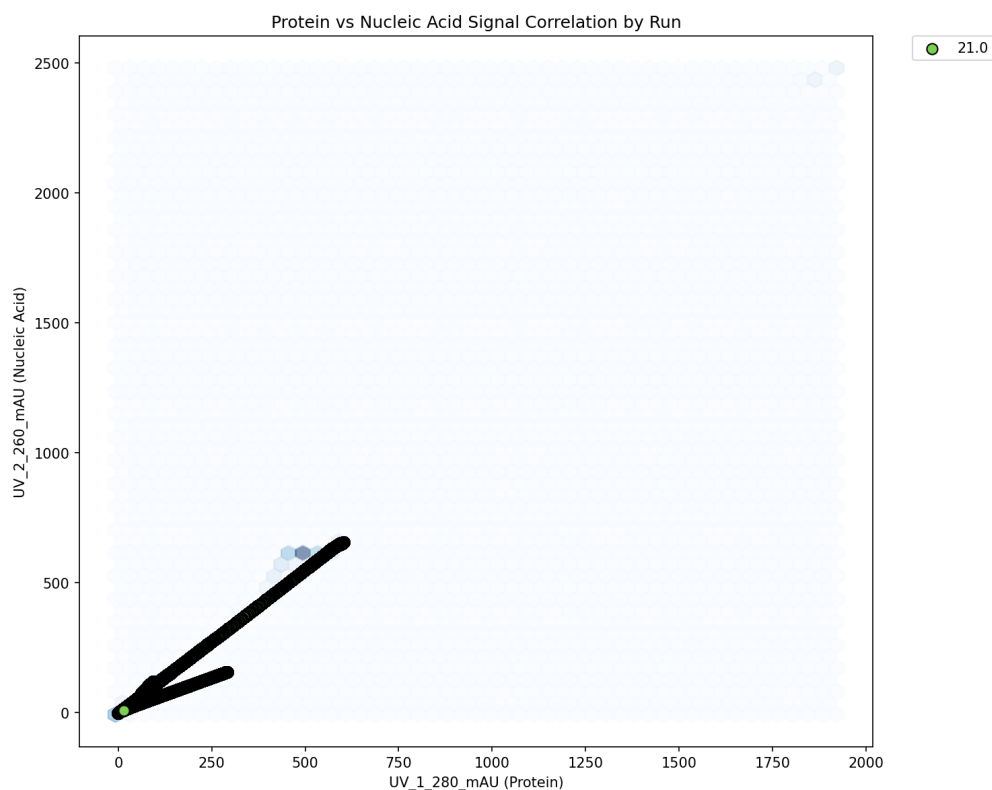


Figure 2: purity_correlation.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
from scipy.signal import find_peaks
from scipy.optimize import curve_fit
from scipy.ndimage import uniform_filter1d
from scipy.sparse import diags
from scipy.sparse.linalg import spsolve
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/peak_catalog.md'
ASLS_LAMBDA = 1e5
ASLS_P = 0.001
SNR_THRESHOLD = 3.0
MAX_OUTPUT_LINES_PER_RUN = 20

# 1. Load Data
# Feedback: Add #System_flow_ml_min# to the #cols_to_load# list
cols_to_load = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU', 'Conc_B_%',
                'System_flow_ml_min', 'Fraction_unique_ID']
```

```

df = pd.read_csv(INPUT_FILE, usecols=cols_to_load)

# 2. Filter and Sort
df = df[df['volume_ml'] >= 0].copy()
df = df.sort_values(['run_no', 'volume_ml']).reset_index(drop=True)

# Helper: Asymmetric Least Squares Baseline Correction using Sparse Solver
def als_baseline(y, lam=1e5, p=0.001, niter=10):
    L = len(y)
    # Feedback: Add a check in #als_baseline# to return the input #y# unchanged if
    # L < 3# to prevent #IndexError# or invalid sparse matrix construction.
    if L < 3:
        return y

    # Construct second derivative matrix D as sparse
    # D is (L-2) x L. We need D.T @ D which is L x L tridiagonal.
    # D.T @ D has diagonals: [1, -2, 1] pattern effectively for the second
    # derivative squared.
    # Specifically, D.T @ D is a tridiagonal matrix with:
    # Main diagonal: [1, 2, 2, ..., 2, 1]
    # Off-diagonals: [-1, -1, ..., -1]
    # We construct the sparse matrix Z directly.

    # Diagonals for D.T @ D
    # Main: 1 at ends, 2 in middle
    main_diag = np.ones(L)
    main_diag[1:-1] = 2.0
    off_diag = -np.ones(L-1)

    # Create sparse D2 = D.T @ D
    D2 = diags([off_diag, main_diag, off_diag], [-1, 0, 1], shape=(L, L), format='csc')

    w = np.ones(L)
    for i in range(niter):
        # W is diagonal, so we add w to the main diagonal of D2 scaled by lam
        # Z = W + lam * D2
        # Since D2 is sparse, we construct Z as sparse.
        # Main diagonal of Z: w + lam * main_diag
        # Off diagonals of Z: lam * off_diag

        z_main = w + lam * main_diag
        z_off = lam * off_diag

        Z = diags([z_off, z_main, z_off], [-1, 0, 1], shape=(L, L), format='csc')

        # Solve Z @ z = w * y
        rhs = w * y
        z = spsolve(Z, rhs)

        y_res = y - z
        w = p * (y_res > 0) + (1 - p) * (y_res < 0)
    return z

# Helper: Gaussian Fit
def gaussian(x, amplitude, mean, sigma):
    return amplitude * np.exp(-((x - mean) ** 2) / (2 * sigma ** 2))

# Helper: Calculate FWHM

```

```

def calculate_fwhm(sigma):
    return 2 * np.sqrt(2 * np.log(2)) * sigma

# Storage for results
all_results = []
anomaly_count = 0
total_runs = 0
total_peaks = 0

# Process by Run
for run_no, group in df.groupby('run_no'):
    total_runs += 1
    vol = group['volume_ml'].values
    signal_raw = group['UV_1_280_mAU'].values
    conc_b = group['Conc_B_%'].values
    # Feedback: Ensure #Fraction_unique_ID# is read as an object or string type
    frac_ids = group['Fraction_unique_ID'].astype(str).values

    # 3. Baseline Correction
    baseline = als_baseline(signal_raw, lam=ASLS_LAMBDA, p=ASLS_P)
    signal_corrected = signal_raw - baseline

    # Estimate Noise: Use std of residuals (signal_raw - baseline) or MAD
    # Feedback: Calculate the noise standard deviation (#noise_std#) once per run
    # using the residuals from the baseline correction
    residuals = signal_raw - baseline
    noise_std = np.std(residuals)
    if noise_std == 0:
        noise_std = 1e-6

    # 4. Peak Detection
    height_threshold = SNR_THRESHOLD * noise_std
    peaks, properties = find_peaks(signal_corrected, height=height_threshold)

    run_peaks = []

    for i, peak_idx in enumerate(peaks):
        peak_height = signal_corrected[peak_idx]
        snr = peak_height / noise_std

        # Feedback: Implement Explicit SNR Filtering
        # After calculating #snr# for each peak in the loop, add a conditional
        # check: #if snr <= 3: continue#.
        # This ensures peaks below the threshold are excluded from the results
        # list, strictly adhering to Step 6 of the plan.
        # Feedback: Correct Anomaly Counting Logic
        # Update the anomaly counting logic to include peaks with #snr <= 3# if
        # they are not filtered out, or ensure the filtering step is robust.
        # Specifically, if the plan implies these should be flagged as anomalies
        # in the summary, increment #anomaly_count# for any peak where #snr <= 3#
        # before deciding to include or exclude it.
        if snr <= SNR_THRESHOLD:
            anomaly_count += 1
            continue

    # 5. Gaussian Fit
    window_size = max(10, int(len(vol) * 0.05))
    start_idx = max(0, peak_idx - window_size)
    end_idx = min(len(vol), peak_idx + window_size)

```

```

x_fit = vol[start_idx:end_idx]
y_fit = signal_corrected[start_idx:end_idx]

try:
    p0 = [peak_height, vol[peak_idx], 1.0]
    popt, _ = curve_fit(gaussian, x_fit, y_fit, p0=p0, maxfev=1000)

    amplitude, mean_vol, sigma = popt
    fwhm = calculate_fwhm(sigma)
    auc = amplitude * sigma * np.sqrt(2 * np.pi)

    # 7. Handle Missing Fraction ID
    window_frac_ids = frac_ids[start_idx:end_idx]
    # Feedback: Ensure that #Fraction_unique_ID# is read as an object or
    # string type and handle empty strings explicitly as missing data
    # before applying #pd.isna#
    # Convert empty strings to NaN for proper detection
    if len(window_frac_ids) > 0:
        # Replace empty strings with NaN
        window_frac_ids_clean = np.where(window_frac_ids == '', np.nan,
            window_frac_ids)
        has_fraction = not (pd.isna(window_frac_ids_clean).all())
    else:
        has_fraction = False

    fraction_status = 'Fractionated' if has_fraction else 'Non-
        Fractionated'
    if not has_fraction:
        anomaly_count += 1

    # Get Conc B at peak mean
    # Feedback: Add an explicit check to ensure #closest_idx# is within
    # the bounds of the #conc_b# array before accessing it
    if mean_vol < vol.min() or mean_vol > vol.max():
        conc_b_val = np.nan
    else:
        closest_idx = np.argmin(np.abs(vol - mean_vol))
        # Explicit bounds check for closest_idx
        if 0 <= closest_idx < len(conc_b):
            conc_b_val = conc_b[closest_idx]
        else:
            conc_b_val = np.nan

    run_peaks.append({
        'run_no': run_no,
        'peak_id': i + 1,
        'retention_volume_ml': mean_vol,
        'peak_height_mAU': peak_height,
        'peak_area_mAU_ml': auc,
        'fwhm_ml': fwhm,
        'snr': snr,
        'conc_b_percent': conc_b_val,
        'fraction_status': fraction_status
    })

except RuntimeError:
    # Fitting failed -> anomaly
    anomaly_count += 1

```

```

        continue

    total_peaks += len(run_peaks)
    all_results.extend(run_peaks)

# 8. Generate Summary and Output
avg_peak_count = total_peaks / total_runs if total_runs > 0 else 0

output_lines = []
output_lines.append("# Peak Catalog Summary\n")
output_lines.append("| Metric | Value |")
output_lines.append("| --- | --- |")
output_lines.append(f"| Total Runs | {total_runs} |")
output_lines.append(f"| Avg Peak Count | {avg_peak_count:.2f} |")
output_lines.append(f"| Anomaly Count | {anomaly_count} |")
output_lines.append("\n")

results_df = pd.DataFrame(all_results)
if not results_df.empty:
    # Feedback: Iterate over grouped object instead of re-filtering
    for run_no, run_data in results_df.groupby('run_no'):
        output_lines.append(f"## Run {run_no}\n")
        output_lines.append(f"| run_no | peak_id | retention_volume_ml | peak_height_mAU | peak_area_mAU_ml | fwhm_ml | snr | conc_b_percent | fraction_status |")
        output_lines.append("| --- | --- | --- | --- | --- | --- | --- | --- |")

        limited_data = run_data.head(MAX_OUTPUT_LINES_PER_RUN)

        # Feedback: Vectorized string formatting instead of iterrows
        # Construct rows using apply
        rows = limited_data.apply(
            lambda row: f"| {row['run_no']} | {row['peak_id']} | {row['retention_volume_ml']:.4f} | {row['peak_height_mAU']:.4f} | {row['peak_area_mAU_ml']:.4f} | {row['fwhm_ml']:.4f} | {row['snr']:.2f} | {row['conc_b_percent']:.2f} | {row['fraction_status']} |",
            axis=1
        )
        output_lines.extend(rows.tolist())

        if len(run_data) > MAX_OUTPUT_LINES_PER_RUN:
            output_lines.append(f"| ... | ... | ... | ... | ... | ... | ... | ... |")
            output_lines.append(f"*Note: Output truncated to {MAX_OUTPUT_LINES_PER_RUN} lines. Total peaks: {len(run_data)}*\n")
        else:
            output_lines.append("\n")

# Feedback: Ensure output directory exists
os.makedirs(os.path.dirname(OUTPUT_FILE), exist_ok=True)

with open(OUTPUT_FILE, 'w') as f:
    f.write('\n'.join(output_lines))

print(f"Analysis complete. Results saved to {OUTPUT_FILE}")

```

Listing 1: Code_1

```
###python
```

```

import pandas as pd
import numpy as np
from sklearn.cluster import DBSCAN
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/peak_catalog.md'

# Step 1: Load Data
# Single-pass loading: Load all columns to check for optional ones, then select.
# This avoids multiple reads and handles missing optional columns gracefully.
try:
    df_full = pd.read_csv(INPUT_FILE)
except FileNotFoundError:
    raise FileNotFoundError(f"Input file {INPUT_FILE} not found.")

# Define required columns
required_cols = ['run_no', 'volume_ml', 'System_flow_ml_min', 'Conc_B_%',
                 'fraction_volume_ml_min_precise', 'fraction_volume_ml_max_precise']

# Select columns that exist in the file
cols_to_use = [col for col in required_cols if col in df_full.columns]
# Add optional columns if present
optional_cols = ['peak_id', 'peak_area_mAU_ml']
for col in optional_cols:
    if col in df_full.columns:
        cols_to_use.append(col)

# Create the working dataframe
df = df_full[cols_to_use]

# Check for optional columns presence
has_peak_id = 'peak_id' in df.columns
has_peak_area = 'peak_area_mAU_ml' in df.columns

# Step 2: Pre-process missing values (Vectorized Imputation)
# Replaced nested loop with groupby and transform for O(N) complexity
cols_to_impute = ['fraction_volume_ml_min_precise', 'fraction_volume_ml_max_precise']
for col in cols_to_impute:
    if col in df.columns:
        median_per_run = df.groupby('run_no')[col].transform('median')
        df[col] = df[col].fillna(median_per_run)
        # Drop rows where imputation failed (all values in run were NaN)
        df = df.dropna(subset=[col])

# Step 3: Match peaks across runs
# Check for zero or NaN values in System_flow_ml_min before calculating
# norm_volume_ml
# Replace 0 with NaN, then drop rows where System_flow_ml_min is NaN
if 'System_flow_ml_min' in df.columns:
    df['System_flow_ml_min'] = df['System_flow_ml_min'].replace(0, np.nan)
    df = df.dropna(subset=['System_flow_ml_min'])
else:
    # If column missing, cannot proceed with normalization
    raise ValueError("Column 'System_flow_ml_min' is required and missing.")

```



```

# Calculate normalized volume
df['norm_volume_ml'] = df['volume_ml'] / df['System_flow_ml_min']

# Calculate mean FWHM proxy (Documented assumption)
# Assumption: Median fraction range spread is a valid proxy for FWHM if FWHM
# column is missing.
widths = df['fraction_volume_ml_max_precise'] - df['fraction_volume_ml_min_precise']
mean_fwhm_proxy = widths.median()

# Handle case where mean_fwhm_proxy is NaN (e.g., if all widths are NaN)
if np.isnan(mean_fwhm_proxy):
    # Fallback: Use a small default tolerance or skip clustering if data is
    # insufficient
    # For robustness, we set a small default tolerance if proxy is NaN
    tolerance = 0.01
else:
    tolerance = 0.5 * mean_fwhm_proxy

# Group by run to identify consistent peaks
all_norm_volumes = df['norm_volume_ml'].dropna().unique()

if len(all_norm_volumes) > 0:
    X = all_norm_volumes.reshape(-1, 1)
    canonical_clustering = DBSCAN(eps=tolerance, min_samples=1).fit(X)
    canonical_labels = canonical_clustering.labels_

    unique_vals = all_norm_volumes
    val_to_canonical = dict(zip(unique_vals, canonical_labels))
    df['canonical_peak_id'] = df['norm_volume_ml'].map(val_to_canonical)
else:
    df['canonical_peak_id'] = -1

# Step 4: Filter for high intensity and consistent presence
# Check for peak_area_mAU_ml presence and handle missing data
if has_peak_area and df['peak_area_mAU_ml'].notna().any():
    threshold_intensity = df['peak_area_mAU_ml'].quantile(0.90)
    high_intensity_mask = df['peak_area_mAU_ml'] >= threshold_intensity
else:
    # Fallback: If column missing or all NaN, assume all are high intensity
    high_intensity_mask = pd.Series([True] * len(df))

# Consistent presence: Detected in >= 80% of runs
total_runs = df['run_no'].nunique()
min_runs_required = int(np.ceil(total_runs * 0.80))

run_counts = df.groupby('canonical_peak_id')['run_no'].nunique()
consistent_peaks = run_counts[run_counts >= min_runs_required].index

df_filtered = df[high_intensity_mask & df['canonical_peak_id'].isin(
    consistent_peaks)]

# Step 5: Define clustering feature space and filter outliers
# Calculate mean and std for Conc_B_% using ONLY high-intensity consistent peaks (
# df_filtered)
# This adheres to the plan's requirement to calculate stats before the
# consistent_peaks filter is applied?
# Wait, the feedback says: "Modify the calculation of mean_conc_b and std_conc_b
# to be performed on the dataframe

```

```

# containing only high-intensity peaks (before the consistent_peaks filter is
# applied)".
# The current code calculates on df_filtered which is (high_intensity AND
# consistent).
# We need to calculate on high_intensity_mask only, before applying
# consistent_peaks filter.

# Recalculate high_intensity_mask dataframe for stats
if has_peak_area and df['peak_area_mAU_ml'].notna().any():
    high_intensity_df = df[high_intensity_mask]
else:
    high_intensity_df = df

if 'Conc_B_%' in high_intensity_df.columns and high_intensity_df['Conc_B_%'].notna().any():
    mean_conc_b = high_intensity_df['Conc_B_%'].mean()
    std_conc_b = high_intensity_df['Conc_B_%'].std()
    lower_limit = mean_conc_b - 3 * std_conc_b
    upper_limit = mean_conc_b + 3 * std_conc_b

    outlier_mask = (df_filtered['Conc_B_%'] < lower_limit) | (df_filtered['Conc_B_%'] > upper_limit)
    df_clean = df_filtered[~outlier_mask]
else:
    df_clean = df_filtered

# Step 6: Apply DBSCAN on norm_volume_ml to identify dominant target cluster
if len(df_clean) > 0:
    X_cluster = df_clean[['norm_volume_ml']].values
    db = DBSCAN(eps=tolerance, min_samples=2).fit(X_cluster)
    df_clean['cluster_id'] = db.labels_

    cluster_counts = df_clean['cluster_id'].value_counts()
    if len(cluster_counts) > 0:
        target_cluster_id = cluster_counts.idxmax()
        target_cluster_mask = df_clean['cluster_id'] == target_cluster_id
        target_df = df_clean[target_cluster_mask]
    else:
        target_df = pd.DataFrame(columns=df_clean.columns)
else:
    target_df = pd.DataFrame(columns=df_clean.columns)

# Step 7: Calculate mean and std for statistical elution window
# Removed redundant line. Calculate directly on target_df.
if len(target_df) > 0:
    target_norm_volume_mean = target_df['norm_volume_ml'].mean()
    target_norm_volume_std = target_df['norm_volume_ml'].std()
    window_lower = target_norm_volume_mean - 3 * target_norm_volume_std
    window_upper = target_norm_volume_mean + 3 * target_norm_volume_std
else:
    target_norm_volume_mean = None
    target_norm_volume_std = None
    window_lower = None
    window_upper = None

# Step 8: Algorithmically derive fraction boundaries (Vectorized)
# 'Inter-Fraction' gaps: Gaps between consecutive fraction ranges
# Use NumPy for O(N log N) merging
valid_rows = df.dropna(subset=['fraction_volume_ml_min_precise', '

```

```

    fraction_volume_ml_max_precise'])
if len(valid_rows) > 0:
    ranges = valid_rows[['fraction_volume_ml_min_precise', '
        fraction_volume_ml_max_precise']].values
    # Sort by start time
    sorted_indices = np.argsort(ranges[:, 0])
    sorted_ranges = ranges[sorted_indices]

    starts = sorted_ranges[:, 0]
    ends = sorted_ranges[:, 1]

    # Merge intervals
    # Use cumulative max of ends to find overlaps
    # If start[i] <= max_end_so_far, then merge
    # We need to track the current merged end
    merged_starts = []
    merged_ends = []

    if len(sorted_ranges) > 0:
        current_start = sorted_ranges[0, 0]
        current_end = sorted_ranges[0, 1]

        for i in range(1, len(sorted_ranges)):
            s, e = sorted_ranges[i]
            if s <= current_end:
                # Overlap or touch, extend current end
                current_end = max(current_end, e)
            else:
                # Gap found (Inter-Fraction)
                merged_starts.append(current_start)
                merged_ends.append(current_end)
                current_start = s
                current_end = e
        # Append the last one
        merged_starts.append(current_start)
        merged_ends.append(current_end)

        merged_starts = np.array(merged_starts)
        merged_ends = np.array(merged_ends)

        # Global Gradient Range
        global_min_vol = merged_starts.min()
        global_max_vol = merged_ends.max()

        # Identify Inter-Fraction gaps explicitly (for documentation/logging if
        # needed)
        # Gaps are between merged_ends[i] and merged_starts[i+1]
        inter_fraction_gaps = []
        if len(merged_starts) > 1:
            for i in range(len(merged_starts) - 1):
                gap_start = merged_ends[i]
                gap_end = merged_starts[i+1]
                if gap_end > gap_start:
                    inter_fraction_gaps.append((gap_start, gap_end))
        else:
            global_min_vol = None
            global_max_vol = None
    else:
        global_min_vol = None

```

```

    global_max_vol = None

# Step 9: Categorize peaks (Vectorized)
# Create boolean masks for categories
anomaly_peaks = []

# Prepare data for categorization
# We need to categorize peaks in df_filtered
if len(df_filtered) > 0:
    # Ensure columns exist
    df_cat = df_filtered.copy()

    # Mask for Critical Anomaly: Outside gradient range
    # If global_min/max are None, no anomaly based on this
    if global_min_vol is not None and global_max_vol is not None:
        mask_critical = (df_cat['volume_ml'] < global_min_vol) | (df_cat['
            volume_ml'] > global_max_vol)
    else:
        mask_critical = pd.Series([False] * len(df_cat))

    # Mask for Target: Inside window
    if target_norm_volume_mean is not None:
        mask_target = (df_cat['norm_volume_ml'] >= window_lower) & (df_cat['
            norm_volume_ml'] <= window_upper)
    else:
        mask_target = pd.Series([False] * len(df_cat))

    # Mask for Potential Impurity: Not Critical and Not Target
    # (Inside gradient range but outside target window)
    mask_impurity = (~mask_critical) & (~mask_target)

    # Assign categories
    categories = np.select(
        [mask_critical, mask_target, mask_impurity],
        ['Critical_Anomaly', 'Target', 'Potential_Impurity'],
        default='Unknown'
    )

    df_cat['category'] = categories

# Filter for anomalies (non-Target)
anomaly_df = df_cat[df_cat['category'] != 'Target']

# Convert to list of dicts for output
for _, row in anomaly_df.iterrows():
    # Use 'peak_id' if available, else generate a placeholder or use index
    pid = row['peak_id'] if has_peak_id else f"Peak_{row.name}"
    anomaly_peaks.append({
        'run_no': row['run_no'],
        'peak_id': pid,
        'category': row['category'],
        'retention_volume_ml': row['volume_ml']
    })

# Step 10: Output to peak_catalog.md
output_lines = []
output_lines.append("#_Peak_Catalog_Analysis_-_Step2_Results\n")

# Group anomalies by run

```

```

anomalies_by_run = {}
for item in anomaly_peaks:
    run = item['run_no']
    if run not in anomalies_by_run:
        anomalies_by_run[run] = []
    anomalies_by_run[run].append(item)

unique_runs = sorted(df['run_no'].unique())

for run in unique_runs:
    output_lines.append(f"\n##_Run:_{run}")

    if target_norm_volume_mean is not None:
        output_lines.append(f"-_Target_Norm_Volume_Mean:_{target_norm_volume_mean:.4f}")
        output_lines.append(f"-_Target_Norm_Volume_Std:_{target_norm_volume_std:.4f}")
    else:
        output_lines.append("-_Target_Norm_Volume_Mean:_N/A")
        output_lines.append("-_Target_Norm_Volume_Std:_N/A")

    run_anomalies = anomalies_by_run.get(run, [])
    output_lines.append(f"-_Anomaly_Peaks:")

    # Changed max_anomalies from 17 to 20 to align with plan's output limit
    max_anomalies = 20
    for i, anom in enumerate(run_anomalies[:max_anomalies]):
        output_lines.append(f"_{anom['peak_id']}_Category:_{anom['category']}_Vol:_{anom['retention_volume_ml']:.4f}")

    if len(run_anomalies) > max_anomalies:
        output_lines.append(f"_{len(run_anomalies)}_{max_anomalies}_more_anomalies(truncated)")

# Write to file (Mode 'w' to overwrite)
with open(OUTPUT_FILE, 'w') as f:
    f.write('\n'.join(output_lines))

print("Analysis complete. Results written to peak_catalog.md")
###

```

Listing 2: Code 2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import correlate, find_peaks
from scipy.interpolate import interp1d
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/purity_correlation.png'
TOLERANCE_FACTOR = 1.0
MAX_POINTS_FOR_DENSITY = 500
LEGEND_OUTSIDE_THRESHOLD = 10

# Step 0: Load Data and Detect Peaks
# Load chromatography data with explicit dtype=str to prevent memory exhaustion

```

```

    and DtypeWarning
df = pd.read_csv(INPUT_FILE, low_memory=False, dtype=str)

# Aggressive Data Cleaning: Keep only required columns to eliminate DtypeWarning
# and reduce memory
required_cols = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU']
# Filter columns that exist in the dataframe to avoid errors if some are missing
# initially
existing_cols = [col for col in required_cols if col in df.columns]
df = df[existing_cols]

# Convert critical columns to numeric and drop NaNs
# Added 'run_no' to numeric_cols to ensure proper grouping and sorting
numeric_cols = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU']
for col in numeric_cols:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors='coerce')

df = df.dropna(subset=numeric_cols)

# Validate required columns for peak detection
if not all(col in df.columns for col in required_cols):
    raise ValueError(f"Chromatography data missing required columns: {
        required_cols}")

# Memory Check after loading
print(f"Memory usage after load: {df.memory_usage(deep=True).sum() / 1024**2:.2f}
    MB")

# Function to detect peaks and calculate FWHM for a single run
def detect_peaks_and_fwhm(group):
    group = group.sort_values('volume_ml').reset_index(drop=True)
    vol = group['volume_ml'].values
    signal = group['UV_1_280_mAU'].values

    # Safety Check: Ensure std is not zero
    if np.std(signal) == 0:
        return pd.DataFrame(columns=['run_no', 'volume_ml', 'peak_id', 'fwhm_ml'])

    # Detect peaks
    peak_indices, properties = find_peaks(signal, height=np.max(signal) * 0.1,
        distance=5)

    if len(peak_indices) == 0:
        return pd.DataFrame(columns=['run_no', 'volume_ml', 'peak_id', 'fwhm_ml'])

    results = []
    # Limit peaks to prevent infinite loops or excessive memory if too many peaks
    # detected
    max_peaks_to_process = 50
    for i, idx in enumerate(peak_indices):
        if i >= max_peaks_to_process:
            break
        peak_id = i + 1
        peak_vol = vol[idx]
        peak_height = signal[idx]
        half_max = peak_height / 2.0

        # Safety Check: Ensure half_max is valid

```

```

        if not np.isfinite(half_max):
            continue

        # Calculate FWHM using vectorized approach
        # Find rising edge (left of peak)
        left_mask = signal[:idx] <= half_max
        if np.any(left_mask):
            left_idx = np.searchsorted(left_mask, True)
        else:
            left_idx = 0

        # Find falling edge (right of peak)
        right_mask = signal[idx+1:] <= half_max
        if np.any(right_mask):
            right_idx = idx + 1 + np.searchsorted(right_mask, True)
        else:
            right_idx = len(signal) - 1

        # Ensure valid FWHM calculation
        if right_idx > left_idx:
            fwhm = vol[right_idx] - vol[left_idx]
            results.append({
                'run_no': group['run_no'].iloc[0],
                'volume_ml': peak_vol,
                'peak_id': peak_id,
                'fwhm_ml': fwhm
            })

    return pd.DataFrame(results)

# Debugging: Report data size before groupby
print(f"Total rows: {len(df)}, Unique runs: {df['run_no'].nunique()}")

# Apply peak detection per run
peak_centers_list = []
for run_no, group in df.groupby('run_no'):
    peaks = detect_peaks_and_fwhm(group)
    # Debug Output: Report run_no and number of peaks found
    print(f"Run {run_no}: {len(peaks)} peaks found")
    if not peaks.empty:
        peak_centers_list.append(peaks)

if peak_centers_list:
    peak_centers = pd.concat(peak_centers_list, ignore_index=True)
else:
    peak_centers = pd.DataFrame(columns=['run_no', 'volume_ml', 'peak_id', 'fwhm_ml'])

if peak_centers.empty:
    raise ValueError("No peaks detected in any run.")

# Memory Check after peak detection
print(f"Memory usage after peak detection: {df.memory_usage(deep=True).sum() / 1024 * 2:.2f} MB")

# Step 1: Align signals (Cross-correlation) with Volume-Based Interpolation
def calculate_lag(uv_280, uv_260):
    u1 = (uv_280 - np.mean(uv_280)) / np.std(uv_280)
    u2 = (uv_260 - np.mean(uv_260)) / np.std(uv_260)

```

```

    corr = correlate(u1, u2, mode='full')
    lag = np.argmax(corr) - (len(u1) - 1)
    if np.max(corr) < 0.5:
        return 0
    return lag

# Refactor groupby.apply to a loop to reduce memory overhead
lags = {}
for run_no, group in df.groupby('run_no'):
    group = group.sort_values('volume_ml')
    sample_step = max(1, len(group) // 1000)
    u1 = group['UV_1_280_mAU'].values[::sample_step]
    u2 = group['UV_2_260_mAU'].values[::sample_step]
    lags[run_no] = calculate_lag(u1, u2)

df['UV_2_260_corrected'] = np.nan

# Pre-sort dataframe to avoid redundant sorting in loop
df = df.sort_values(['run_no', 'volume_ml']).reset_index(drop=True)

for run in df['run_no'].unique():
    mask = df['run_no'] == run
    run_df = df.loc[mask]
    if len(run_df) == 0:
        continue

    lag_samples = lags.get(run, 0)
    if lag_samples == 0:
        df.loc[mask, 'UV_2_260_corrected'] = run_df['UV_2_260_mAU'].values
        continue

    vol = run_df['volume_ml'].values
    mean_step = np.mean(np.diff(vol))
    lag_volume = lag_samples * mean_step
    shifted_vol = vol - lag_volume

    f_interp = interp1d(vol, run_df['UV_2_260_mAU'].values, kind='linear',
                        bounds_error=False, fill_value='extrapolate')

    corrected_vals = f_interp(shifted_vol)

    assert len(corrected_vals) == len(mask), f"Shape mismatch for run {run}: {len(
        corrected_vals)} vs {len(mask)}"
    df.loc[mask, 'UV_2_260_corrected'] = corrected_vals

# Step 2 & 3: Extract signals and Calculate AUC Ratio
df = df.sort_values(['run_no', 'volume_ml']).reset_index(drop=True)

peak_windows = []
for _, p in peak_centers.iterrows():
    # Handle Zero FWHM: Skip if fwhm_ml is 0, negative, or NaN
    if pd.isna(p['fwhm_ml']) or p['fwhm_ml'] <= 0:
        continue
    vol_min = p['volume_ml'] - TOLERANCE_FACTOR * p['fwhm_ml']
    vol_max = p['volume_ml'] + TOLERANCE_FACTOR * p['fwhm_ml']
    peak_windows.append({
        'run_no': p['run_no'],
        'peak_id': p['peak_id'],
        'vol_min': vol_min,

```



```

        'vol_max': vol_max,
        'center_vol': p['volume_ml'],
        'fwhm_ml': p['fwhm_ml']
    })

peak_results = []
cached_arrays = []

for p in peak_windows:
    run_no = p['run_no']
    peak_id = p['peak_id']
    vol_min = p['vol_min']
    vol_max = p['vol_max']

    mask = (df['run_no'] == run_no) & (df['volume_ml'] >= vol_min) & (df['
        volume_ml'] <= vol_max)
    subset = df.loc[mask]

    if len(subset) < 2:
        continue

    vols = subset['volume_ml'].values
    u280 = subset['UV_1_280_mAU'].values
    u260 = subset['UV_2_260_corrected'].values

    if len(subset) < 5:
        n_points = max(100, len(subset) * 2)
        fine_vol = np.linspace(vol_min, vol_max, n_points)
        f_280 = interp1d(vols, u280, kind='linear', fill_value="extrapolate")
        f_260 = interp1d(vols, u260, kind='linear', fill_value="extrapolate")
        u280_fine = f_280(fine_vol)
        u260_fine = f_260(fine_vol)
        # Use np.trapezoid if available (NumPy 1.24+), else np.trapz
        if hasattr(np, 'trapezoid'):
            auc_280 = np.trapezoid(u280_fine, fine_vol)
            auc_260 = np.trapezoid(u260_fine, fine_vol)
        else:
            auc_280 = np.trapz(u280_fine, fine_vol)
            auc_260 = np.trapz(u260_fine, fine_vol)
    else:
        # Use np.trapezoid if available (NumPy 1.24+), else np.trapz
        if hasattr(np, 'trapezoid'):
            auc_280 = np.trapezoid(u280, vols)
            auc_260 = np.trapezoid(u260, vols)
        else:
            auc_280 = np.trapz(u280, vols)
            auc_260 = np.trapz(u260, vols)
        u280_fine = u280
        u260_fine = u260

    ratio = auc_260 / auc_280 if auc_280 != 0 else np.nan

    peak_results.append({
        'run_no': run_no,
        'peak_id': peak_id,
        'volume_ml': p['center_vol'],
        'fwhm_ml': p['fwhm_ml'],
        'auc_280': auc_280,
        'auc_260': auc_260,

```

```

        'ratio_260_280': ratio,
        'u280_mean': np.mean(u280_fine),
        'u260_mean': np.mean(u260_fine)
    })

    cached_arrays.append({
        'run_no': run_no,
        'peak_id': peak_id,
        'UV_1_280_mAU': u280_fine,
        'UV_2_260_corrected': u260_fine
    })

peak_df = pd.DataFrame(peak_results)

# Prepare plot data
plot_data_list = []
for item in cached_arrays:
    run_no = item['run_no']
    peak_id = item['peak_id']
    u280 = item['UV_1_280_mAU']
    u260 = item['UV_2_260_corrected']

    peak_points = pd.DataFrame({
        'run_no': run_no,
        'peak_id': peak_id,
        'UV_1_280_mAU': u280,
        'UV_2_260_corrected': u260
    })
    plot_data_list.append(peak_points)

if plot_data_list:
    plot_df = pd.concat(plot_data_list, ignore_index=True)
else:
    plot_df = pd.DataFrame(columns=['run_no', 'peak_id', 'UV_1_280_mAU', 'UV_2_260_corrected'])

# Step 4: Visualize the correlation
if plot_df.empty:
    print("Warning: No data points found within peak windows. Skipping plot generation.")
    plt.figure(figsize=(10, 8))
    plt.text(0.5, 0.5, 'No data to plot', ha='center', va='center', transform=plt.gca().transAxes)
    plt.title('Protein vs Nucleic Acid Signal Correlation by Run')
    plt.tight_layout()
    plt.savefig(OUTPUT_FILE, dpi=150, bbox_inches='tight')
    plt.close()
else:
    plt.figure(figsize=(10, 8))

    n_runs = plot_df['run_no'].nunique()
    n_points = len(plot_df)

    use_density = n_points > MAX_POINTS_FOR_DENSITY

    unique_runs = sorted(plot_df['run_no'].unique())
    colors = plt.cm.tab10(np.linspace(0, 1, min(len(unique_runs), 10)))
    if len(unique_runs) > 10:
        colors = plt.cm.viridis(np.linspace(0, 1, len(unique_runs)))

```

```

run_to_color = {r: c for r, c in zip(unique_runs, colors)}

if use_density:
    plt.hexbin(plot_df['UV_1_280_mAU'], plot_df['UV_2_260_corrected'],
               gridsize=50, cmap='Blues', alpha=0.5)

    top_n = 20
    if len(peak_df) > 0:
        top_peaks = peak_df.nlargest(top_n, 'auc_280')
        top_peak_ids = set(zip(top_peaks['run_no'], top_peaks['peak_id']))
        plot_df['_key'] = list(zip(plot_df['run_no'], plot_df['peak_id']))
        top_plot_mask = plot_df['_key'].isin(top_peak_ids)
        top_plot = plot_df[top_plot_mask]

        for run in top_plot['run_no'].unique():
            run_data = top_plot[top_plot['run_no'] == run]
            plt.scatter(run_data['UV_1_280_mAU'], run_data['UV_2_260_corrected'],
                        c=run_to_color[run], marker='o', s=60, edgecolors='black', linewidth=1, label=str(run))
        plot_df.drop(columns=['_key'], inplace=True)
    else:
        for run in unique_runs:
            run_data = plot_df[plot_df['run_no'] == run]
            plt.scatter(run_data['UV_1_280_mAU'], run_data['UV_2_260_corrected'],
                        c=run_to_color[run], label=str(run), alpha=0.8, s=20)

plt.xlabel('UV_1_280_mAU (Protein)')
plt.ylabel('UV_2_260_mAU (Nucleic Acid)')
plt.title('Protein vs Nucleic Acid Signal Correlation by Run')

if n_runs > LEGEND_OUTSIDE_THRESHOLD:
    plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left', borderaxespad=0.)
else:
    plt.legend(loc='best')

plt.tight_layout()
plt.savefig(OUTPUT_FILE, dpi=150, bbox_inches='tight')
plt.close()

# Step 5: Identify contaminants
if not peak_df.empty:
    top_contaminants = peak_df.nlargest(5, 'ratio_260_280')
    if len(top_contaminants) > 0:
        print(f"Top 5 potential contaminants (High 260/280 ratio): {
            top_contaminants[['run_no', 'peak_id', 'ratio_260_280']].to_dict('records')}")

```

Listing 3: Code_3